

Group Assignment #1

Mini Data Warehouse & Analytics Dashboard

กรณีศึกษา: ธุรกิจร้านพิซซ่า ปีงบประมาณ 2015

รหัสผลิตภัณฑ์	ชื่อ-สกุล	หน้าที่รับผิดชอบ
		Business Requirements
		ETL & Data Warehouse
		Dashboard & Insights

1. บทนำและบริบททางธุรกิจ

1.1 ปัญหาทางธุรกิจ

ร้านพิซซาคาริศึกษาเปิดให้บริการตลอดปี 2015 มีรายการขายสะสมในระบบ POS จำนวน **21,350 ออเดอร์** คิดเป็น **48,620 รายการสินค้า** จากเมนู **32 ชนิด** ที่แตกออกเป็น **96 SKU** (4 หมวด × 5 ขนาด) แต่ข้อมูลทั้งหมดถูกใช้เพียงเพื่อปัดยอดขายรายวันเท่านั้น ไม่เคยถูกนำมาวิเคราะห์เพื่อการตัดสินใจ ผลที่ตามมาคือการบริหารร้านยังอาศัยประสบการณ์ของผู้จัดการเป็นหลัก และเกิดปัญหา 4 ด้าน

- ไม่ทราบว่ามีเมนูใดควรคงไว้หรือควรตัดออก** — เมนู 96 SKU สร้างภาระทั้งด้าน การจัดเก็บวัตถุดิบและความเร็วในการผลิต แต่ไม่มีข้อมูลว่ารายการใดขายไม่ออก
- จัดกะพนักงานไม่ตรงกับความต้องการจริง** — ระบบบันทึกเวลาสั่งซื้อละเอียด ถึงระดับวินาที แต่ไม่เคยนำมาหาชั่วโมงเร่งด่วน
- วางแผนวัตถุดิบด้วยการคาดเดา** — เกิดทั้งของเสียจากการสั่งเกิน และการเสียโอกาสขายจากของขาด
- อธิบายยอดขายที่ผันผวนไม่ได้** — แยกไม่ออกว่าเกิดจากปัจจัยภายในหรือภายนอก เมื่ออธิบายสาเหตุไม่ได้ก็พยากรณ์และวางแผนล่วงหน้าไม่ได้

ช่องว่างสำคัญ — ข้อมูล POS เพียงลำพังตอบได้แค่ “ขายอะไรไปเท่าไร” แต่ตอบไม่ได้ว่า “ทำไม” โครงการนี้จึงต้องนำข้อมูลภายนอกเข้ามาเชื่อมโยงด้วย

1.2 ผู้ใช้งานข้อมูล (Stakeholders)

ผู้ใช้งาน	ต้องการทราบ	นำไปตัดสินใจ
เจ้าของร้าน / ผู้จัดการ	ภาพรวมยอดขาย แนวโน้มรายเดือน เมนูทำเงิน	ปรับกลยุทธ์ ตั้งเป้าหมาย ตัดสินใจลงทุน
หัวหน้าครัว / ฝ่ายจัดซื้อ	ปริมาณขายแยกตามเมนูและขนาด รูปแบบตามฤดูกาล	วางแผนสั่งวัตถุดิบ ลดของเสีย
หัวหน้ากะ	ชั่วโมงและวันที่มีออเดอร์หนาแน่น	จัดตารางเวร กระจายกำลังคน
ฝ่ายการตลาด	ช่วงเวลาที่ยอดตก เมนูที่ขายไม่ออก ผลของวันหยุด	ออกโปรโมชั่น เลือกจังหวะจัดแคมเปญ

1.3 วัตถุประสงค์

วัตถุประสงค์หลัก: พัฒนา Mini Data Warehouse และ Interactive Dashboard ที่รวมข้อมูลยอดขายจากระบบ POS เข้ากับข้อมูลสภาพอากาศและปฏิทินวันหยุด เพื่อเปลี่ยนข้อมูลดิบที่ถูกทิ้งไว้ให้กลายเป็นเครื่องมือสนับสนุนการตัดสินใจ ด้านเมนู กำลังคน และการวางแผนวัตถุดิบ ที่ผู้บริหารร้านใช้งานได้เอง

1.4 ขอบเขตและข้อจำกัด

อยู่ในขอบเขต

- ยอดขายทั้งหมดของปี 2015 (1 ม.ค. – 31 ธ.ค.)
- วิเคราะห์ระดับรายการสินค้าในบิล เมนู หมวดหมู่ ขนาด ช่วงเวลา
- ความสัมพันธ์ระหว่างยอดขายกับสภาพอากาศและวันหยุด

อยู่นอกขอบเขต

- วิเคราะห์รายลูกค้า — POS ไม่บันทึกรหัสลูกค้า
- เปรียบเทียบระหว่างสาขา — ข้อมูลมาจากสาขาเดียว
- กำไรขั้นต้น — ข้อมูลต้นทุนไม่มีต้นทุนวัตถุดิบ

ข้อจำกัดที่ต้องระบุ — ข้อมูลต้นทางไม่ระบุที่ตั้งร้าน โครงการกำหนดให้ร้านตั้งอยู่ที่ **ชิคาโก** เพื่อให้ดึงข้อมูลสภาพอากาศได้ ผลการวิเคราะห์ด้านอากาศจึงอยู่บนสมมติฐานนี้

2. Business Questions และ Measures

2.1 Business Questions

#	คำถามทางธุรกิจ	ผู้ถาม	Measure	กราฟที่ใช้ตอบ
BQ1	เมนูใดสร้างรายได้สูงสุดและต่ำสุด ควรตัดเมนูใดออก	เจ้าของร้าน / จัดซื้อ	M1, M2	แท่งแนวนอนเรียงลำดับ
BQ2	ขนาดใดเป็นตัวหารรายได้หลัก ขนาด XL/XXL คู่้มที่จะเก็บไว้หรือไม่	จัดซื้อ / หัวหน้าครัว	M1, M2	โดนัท / แท่งซ้อน
BQ3	ช่วงเวลาใดของวันมีลูกค้าหนาแน่นที่สุด ควรจัดกำลังคนอย่างไร	หัวหน้ากะ	M3, M1	กราฟพื้นที่รายชั่วโมง
BQ4	ยอดขายเปลี่ยนแปลงอย่างไรตลอดปี มีฤดูกาลหรือไม่	เจ้าของร้าน / การตลาด	M1, M6	กราฟเส้นตามช่วงเวลา
BQ5	สภาพอากาศมีผลต่อยอดขายหรือไม่ มากพอจะใช้วางแผนไหม	ผู้จัดการ / หัวหน้ากะ	M1, M3, M4	แท่งเปรียบเทียบ
BQ6	วันหยุดนักขัตฤกษ์ทำให้ยอดขายเพิ่มหรือลด	การตลาด / เจ้าของร้าน	M6, M3	แท่งเทียบกลุ่ม
BQ7	หมวดพิซซ่าได้รับความนิยมสูงสุด สมดุลเมนูเหมาะสมหรือไม่	เจ้าของร้าน / ครัว	M1, M2	โดนัทสัดส่วน
BQ8	เหตุใดร้านจึงปิดทำการ 7 วัน และกระทบรายได้เท่าไร	เจ้าของร้าน	M6	ปฏิทิน Heatmap

2.2 Measures

แบ่งเป็น 2 ระดับตามหลัก Dimensional Modeling — **Base Measure** เก็บจริงในคอลัมน์ของ Fact Table บวกกันได้ทุกมิติ และ **Derived Measure** ที่คำนวณตอนแสดงผลบน BI Tool เพราะเป็นอัตราส่วนซึ่งนำมาบวกกันไม่ได้

เหตุผลที่ต้องแยก — หากเก็บ “ยอดขายเฉลี่ยต่อบิล” ลงใน Fact แล้วผู้ใช้ลากไปรวมทั้งเดือน ระบบจะนำค่าเฉลี่ยมาบวกกัน ซึ่งไม่มีความหมายทางคณิตศาสตร์ จึงต้องเก็บเฉพาะตัวตั้งและตัวหารแยกกัน

รหัส	Measure	วิธีคำนวณ	ความหมายและประโยชน์ทางธุรกิจ	ค่าปี 2015
M1	Total Revenue Base	SUM(quantity x unit_price)	มูลค่าขายรวมก่อนหักต้นทุน เป็นตัวชี้วัดหลักที่ผู้บริหารใช้ติดตามผลประกอบการ และเป็นตัวตั้งของอัตราส่วนเกือบทุกตัว	\$817,860.05
M2	Total Quantity Sold Base	SUM(quantity)	จำนวนภาตที่ขายออกไป ใช้วางแผนวัตถุดิบและกำลังการผลิตในครัว ซึ่งสนใจ “ก่ภาค” ไม่ใช่ “ก่บาท”	49,574 ภาค
M3	Number of Orders Base (distinct)	COUNT(DISTINCT order_id)	จำนวนบิล เป็นตัวแทนจำนวนครั้งที่ลูกค้าใช้บริการ ใช้วัดปริมาณงานหน้าร้านและเป็นฐานจัดกะพนักงาน	21,350 บิล
M4	Average Order Value Derived	M1 ÷ M3	ลูกค้าหนึ่งรายจ่ายเฉลี่ยเท่าไร ใช้ประเมินผลการเชียร์ขายเพิ่ม หากออกโปรฯ ขายพวงแล้วตัวเลขไม่ขยับแปลว่าโปรฯ ไม่ได้ผล	\$38.31
M5	Items per Order Derived	M2 ÷ M3	บอกขนาดกลุ่มลูกค้า ค่าสูงแปลว่าเป็นกลุ่มครอบครัวหรือสั่งเข้าออฟฟิศ ซึ่งใช้เวลาผลิตนานกว่า	2.32 ภาค
M6	Revenue per Trading Day Derived	M1 ÷ จำนวนวันทำการ	เทียบผลงานระหว่างเดือนได้อย่างเป็นธรรม เพราะแต่ละเดือนมีวันทำการไม่เท่ากัน การเทียบยอดรวมดิบจะทำให้เดือนที่ร้านปิดหลายวันดูแย่เกินจริง	\$2,284.53
M7	Gross Profit / Margin % Derived — ยังไม่พร้อมใช้	M1 - SUM(qty x cost)	เมนูที่ขายดีที่สุดในจำเป็นต้องทำกำไรดีที่สุด เป็น Measure เดียวที่ตอบได้ว่าควรดันเมนูใด <i>ข้อมูลต้นทางไม่มีต้นทุน จึงยังคำนวณไม่ได้</i>	—

3. แหล่งที่มาและรายละเอียดของข้อมูล

โครงการใช้ข้อมูล 3 แหล่ง ที่มีรูปแบบแตกต่างกัน โดย 2 แหล่งเป็น **Nested JSON** ที่ดึงสดผ่าน **REST API** ซึ่งมีความซับซ้อนมากกว่า CSV แบบตารางทั่วไป

#	แหล่งข้อมูล	รูปแบบ	ปริมาณ	การนำเข้า
1	Pizza Place Sales Maven Analytics	CSV เชิงสัมพันธ์ 4 ตาราง	48,620 รายการ 21,350 ปีล	อ่านไฟล์ในเครื่อง
2	Open-Meteo Historical Weather Archive	Nested JSON ผ่าน REST API	8,760 ชั่วโมง 365 วัน	<code>fetch_sources.py</code> ดึงสด ไม่ต้องใช้ API key
3	Nager.Date Public Holidays API	Nested JSON ผ่าน REST API	16 รายการ 13 วันที่	<code>fetch_sources.py</code> ดึงสด ไม่ต้องใช้ API key
–	WMO Weather Code 4677 ตารางอ้างอิงประกอบ	JSON lookup	28 รหัส	คัดจากเอกสาร Open-Meteo

3.1 เหตุผลในการเลือกแหล่งข้อมูลเพิ่มเติม

ข้อมูล POS ตอบได้เพียงว่าขายอะไรไปเท่าไร การเพิ่มข้อมูลสภาพอากาศรายชั่วโมง และปฏิทินวันหยุดทำให้สามารถตอบคำถามเชิงสาเหตุได้ว่ายอดขายที่ผันผวน เกิดจากปัจจัยภายนอกหรือไม่ ซึ่งเป็นคำถามที่มีมูลค่าทางธุรกิจสูงสุดของโครงการ และเป็นสิ่งที่ข้อมูลชุดเดิมตอบไม่ได้เลย

3.2 การตรวจสอบความเชื่อมโยงระหว่างแหล่งข้อมูล

การเชื่อมโยง	คีย์ที่ใช้	ผลการตรวจสอบ
orders → weather รายชั่วโมง	วันที่ + ชั่วโมง	21,350 / 21,350 (100%) ไม่มีออเดอร์ใดหาสภาพอากาศไม่พบ
ความครบถ้วนของข้อมูลอากาศ	—	8,760 ชั่วโมง = 365 × 24 ครบพอดี ไม่มีชั่วโมงหายและไม่ซ้ำ
ค่าว่างในไฟล์สภาพอากาศ	—	0 ทุกตัวแปร ทั้งรายชั่วโมงและรายวัน
order_details → orders / pizzas	order_id , pizza_id	ไม่มีคีย์ค่าพรา้ เชื่อมได้ครบ 100%
holidays → dim_date	date	ต้อง deduplicate ก่อน มิฉะนั้น Fact จะบานปลาย (ดูหัวข้อ 4)

การยืนยันข้ามแหล่งข้อมูล — วันที่ 25 ธันวาคม 2015 ซึ่งไม่มีบิลขายเลยในข้อมูล POS ตรงกับ *Christmas Day* ในไฟล์วันหยุดจาก Nager.Date พอดี เป็นหลักฐานว่าข้อมูลสองแหล่งที่มาจากคนละระบบสอดคล้องกันจริง

4. ปัญหาคุณภาพข้อมูลและวิธีแก้ไข

จากการ Profile ข้อมูลทั้ง 3 แหล่ง พบปัญหาคุณภาพ **7 ประเภท** ทุกข้อเป็นปัญหาที่มีอยู่จริงในข้อมูลต้นทาง ไม่ได้สร้างขึ้นเอง และทุกข้อบันทึกจำนวนแถวก่อน-หลังการแก้ไขไว้ใน Audit Log ของ Notebook

#	ประเภทปัญหา	รายละเอียดที่พบ	วิธีแก้ไข
1	Encoding ไม่ใช่ UTF-8	pizza_types.csv มี byte 0x91 (smart quote แบบ Windows-1252) ในชื่อวัตถุดิบ 'Nduja Salami' ทำให้อ่านด้วย UTF-8 แล้วเกิด UnicodeDecodeError ทันที	ระบุ encoding='cp1252' ตอนอ่านไฟล์
2	โครงสร้าง Columnar	Open-Meteo คืนค่าเป็น array คู่ขนาน (dict-of-arrays) ไม่ใช่ array ของ record ทำให้โหลดเข้า DW ตรง ๆ ไม่ได้	Transpose เป็น row-oriented ด้วย <code>pd.DataFrame()</code> ได้ 8,760 แถว
3	หน่วยวัดไม่ตรงกัน	API คืนระบบเมตริก (°C, mm, cm, km/h) แต่ธุรกิจอยู่ในสหรัฐฯ ที่ใช้หน่วยอังกฤษ ที่สำคัญคือฝนเป็น mm แต่หิมะเป็น cm ในไฟล์เดียวกัน	แปลงเป็น °F, inch, mph โดยหิมะใช้ตัวหาร 2.54 แยกจากฝนที่ใช้ 25.4
4	รูปแบบวันที่ไม่ตรงกัน	3 แหล่งใช้ 3 รูปแบบ — <code>date + time</code> แยกคอลัมน์ / <code>2015-01-01T00:00</code> / <code>2015-01-01</code>	ระบุ format string ชัดเจนใน <code>pd.to_datetime()</code> ทุกจุด ไม่ปล่อยให้ระบบเดา
5	ข้อมูลซ้ำ อันตรายที่สุด	วันหยุดมี 16 แถวแต่มีเพียง 13 วันที่ — <i>Good Friday</i> ซ้ำ 2 แถว, <i>Columbus Day</i> ซ้ำ 2 แถว และ <i>Indigenous Peoples' Day</i> ซ้อนวันเดียวกัน	ยุบเหลือ 1 แถวต่อวัน โดยให้ <code>global=True</code> มาก่อน
6	ค่าที่เป็นรหัส	<code>weather_code</code> เป็นตัวเลข WMO 4677 — <code>0</code> คือฟ้าใส <code>73</code> คือหิมะตกปานกลาง ผู้ใช้อ่านไม่เข้าใจ	Join กับตารางอ้างอิง WMO ถอดเป็นกลุ่มสภาพอากาศที่อ่านได้
7	Timezone ไม่ขยับตาม DST	ขอข้อมูลด้วย <code>timezone=America/Chicago</code> แต่ API ตอบ <code>utc_offset</code> คงที่ทั้งปี ทั้งที่ซีกโลกเปลี่ยนระหว่าง CST และ CDT	ตัดสินใจไม่แก้ แต่บันทึกเป็นข้อจำกัด (ดูกล่องด้านล่าง)

4.1 การพิสูจน์ผลกระทบของข้อมูลซ้ำ

ปัญหาข้อ 5 เป็นข้อที่อันตรายที่สุด เพราะหาก join เข้า Fact Table โดยไม่แก้ก่อน ยอดขายจะถูกนับซ้ำโดยไม่มีข้อความแจ้งเตือนใด ๆ Notebook จึงสาธิตให้เห็นก่อนว่าปัญหามีอยู่จริง

การทดสอบ	จำนวนแถว
จำนวนออเดอร์ตั้งต้น	21,350
หลัง join ตารางวันหยุดโดยไม่ deduplicate	21,414
แถวที่ออกขึ้นมาโดยไม่มีการแจ้งเตือน	+64

กรณีที่เลือกไม่แก้ไข (ปัญหาข้อ 7) — การวิเคราะห์ของโครงการใช้ช่วงเวลากว้างระดับ *daypart* (มื่อกลางวัน / มื่อเย็น) ความคลาดเคลื่อน 1 ชั่วโมงในช่วงฤดูหนาวจึงไม่ทำให้ข้อสรุปเปลี่ยน ขณะที่การดัดแปลงข้อมูลต้นทางกลับสร้างความเสี่ยงมากกว่าประโยชน์ การตัดสินใจไม่แก้พร้อมเหตุผลและการบันทึกไว้ถือเป็นการจัดการคุณภาพข้อมูลเช่นกัน

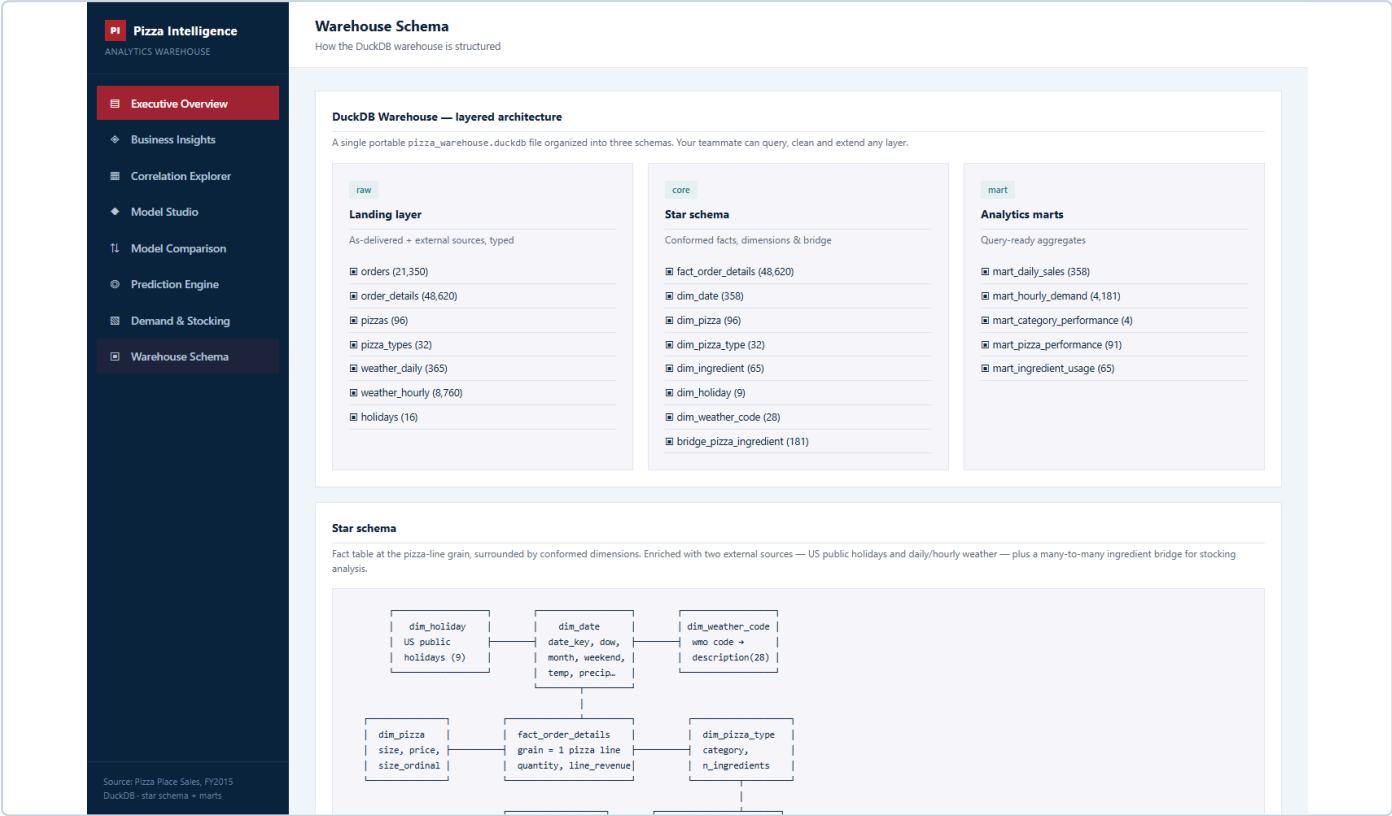
5. การออกแบบ Data Warehouse

5.1 นิยาม Fact Table และ Grain

เหตุการณ์ที่บันทึก: การขายสินค้าหนึ่งรายการในใบเสร็จ ณ เวลาที่ลูกค้าสั่ง จัดเป็น **Transaction Fact Table**

Grain: หนึ่งแถวใน `fact_sales_line` = **พิซซ่าหนึ่งชนิดในหนึ่งขนาด ที่ปรากฏในหนึ่งบิล**

พิซซ่าชนิดและขนาดเดียวกันที่สั่งหลายแถวในบิลเดียว จะรวมเป็นแถวเดียวโดยเก็บจำนวนไว้ในคอลัมน์ `quantity` ตามที่ระบบ POS บันทึกมา



ภาพที่ 1 — โครงสร้าง Star Schema และเส้นทางข้อมูลจาก Raw → ETL → Warehouse (หน้า Warehouse Schema บน Dashboard)

5.2 ตารางในคลังข้อมูล

ตาราง	แถว	Primary Key	คำอธิบาย
fact_sales_line	48,620	sales_key	Fact หลัก มี FK 4 ตัว (date_key , time_key , pizza_key , weather_key) และ Degenerate Dimension order_id สำหรับนับจำนวนบิล Measures: quantity , unit_price , line_revenue
dim_date	365	date_key	ปฏิทินเต็มปี รวมวันที่ร้านเปิด พร้อม is_holiday , holiday_name , is_trading_day , is_weekend , season , quarter
dim_time	24	time_key	ชั่วโมงของวัน พร้อม daypart (Morning / Lunch / Afternoon / Dinner / Late Night)
dim_pizza	96	pizza_key	เมนู × ขนาด แบนราบ รวมชื่อเมนู หมวดหมู และขนาดไว้ในตารางเดียว
dim_weather	23	weather_key	Mini-Dimension จากการจัดสภาพอากาศเป็นช่วง
dim_ingredient	65	ingredient_key	วัตถุดิบแต่ละชนิด พร้อมหมวด (Cheese / Meat / Sauce / Seafood / Vegetable)
bridge_pizza_ingredient	181	คีย์ผสม	ตารางเชื่อมความสัมพันธ์แบบหลายต่อหลายระหว่างเมนูกับวัตถุดิบ

5.3 การตัดสินใจเชิงออกแบบที่สำคัญ

ก. เหตุใด `dim_date` จึงมี 365 แถว ไม่ใช่ 358 แถว

ร้านเปิดขายจริงเพียง 358 วัน หากสร้าง Dimension จากวันที่ที่มียอดขายเท่านั้น วันที่ร้านปิด 7 วันจะหายไปจากมิติ ทำให้ Dashboard ไม่สามารถแสดงได้ว่าวันนั้นขายได้ศูนย์ และตอบ BQ8 ไม่ได้ จึงต้องสร้างจากปฏิทินเต็มปีแล้วใช้ธง `is_trading_day` แยกแทน

ข. เหตุใดจึงรวม `pizzas` กับ `pizza_types` เป็นตารางเดียว

ข้อมูลต้นทางแยกสองตารางซึ่งเป็นโครงสร้างแบบ **Snowflake** การออกแบบ DW จึงรวมเป็น `dim_pizza` ตารางเดียวเพื่อให้เป็น **Star Schema** แต่ ผู้ใช้ BI Tool จะลากใช้ `category` ได้ทันทีโดยไม่ต้อง join ผ่านตารางกลาง

ค. เหตุใดสภาพอากาศจึงเป็น **Mini-Dimension**

อุณหภูมิเป็นค่าต่อเนื่องที่มีค่าไม่ซ้ำกันนับพันค่า หากใส่ลง Dimension โดยตรง จะได้ตารางที่ใหญ่พอ ๆ กับ Fact จึงใช้วิธีจัดค่าเป็นช่วง (banding) แล้วเก็บเฉพาะ ชุดค่าที่เกิดขึ้นจริง ทำให้เหลือเพียง **23 แถวจาก 280 combination ที่เป็นไปได้**

ข้อควรระวังในการใช้ Bridge Table — การ join Fact ผ่าน `bridge_pizza_ingredient` จะทำให้ยอดขายถูกนับซ้ำ เท่ากับจำนวนวัตถุดิบในเมนูนั้น ตารางนี้จึงมีไว้ตอบคำถามเรื่องวัตถุดิบเท่านั้น **ห้ามนำมารวมยอดขาย**

6. กระบวนการ ETL

6.1 เครื่องมือที่ใช้

เครื่องมือ	ใช้ในขั้นตอน	เหตุผลที่เลือก
Python + pandas	ทุกขั้นตอน	จัดการ Nested JSON และ CSV ได้ในเครื่องมือเดียว เขียนเป็นโค้ดจึงรันซ้ำได้
Jupyter Notebook	เอกสารประกอบ ETL	แสดงหลักฐานก่อน-หลังการแก้ไขในไฟล์เดียวกับโค้ด ตรวจสอบย้อนกลับได้
DuckDB	Load และ Query	ฐานข้อมูลแบบคอลัมน์ที่ออกแบบมาเพื่องานวิเคราะห์ เก็บทั้งคลังในไฟล์เดียว ไม่ต้องติดตั้ง server
urllib (stdlib)	Extract จาก API	ไม่เพิ่ม dependency ภายนอก พร้อมกลไก retry แบบ exponential backoff
Chart.js	Dashboard	สร้าง Dashboard เชิงโต้ตอบเป็นไฟล์ HTML เดียว เปิดด้วยเบราว์เซอร์ได้ทันที

6.2 ขั้นตอนการทำงาน

ขั้นตอน	สิ่งที่ทำ	ผลลัพธ์และการตรวจสอบ
Extract	ดึงข้อมูลจาก 2 REST API เก็บเป็น raw JSON และอ่าน CSV 4 ไฟล์	บันทึก manifest ที่มีเวลาดึง URL ขนาดไฟล์ และ SHA-256 ของทุกไฟล์
Clean	ตรวจจับและแก้ปัญหาคอนภาพ 7 ประเภท	ทุกข้อบันทึกจำนวนก่อน-หลังลง Audit Log พร้อม Range Check ค่าที่แปลงหน่วยแล้ว
Transform	สร้าง Dimension 5 ตาราง และ Bridge 1 ตาราง	ตรวจจำนวนแถวและความไม่ซ้ำของ Primary Key ทุกตาราง
Integrate	เชื่อม 3 แหล่งเป็น Fact Table และคำนวณ Measures	ทุก merge ใช้ validate="many_to_one" และตรวจว่าจำนวนแถวยังเป็น 48,620
Load	โหลดเข้า DuckDB แบบ Full Refresh และ export CSV	ลบไฟล์เดิมทั้งหมดทุกครั้ง จึงไม่มีข้อมูลค้างจากรอบก่อน
Validate	เทียบ Control Totals และตรวจ Referential Integrity	คำนวณจากไฟล์ต้นทางโดยตรงแล้วเทียบกับที่ query จากคลัง

6.3 การตรวจสอบความถูกต้องก่อนและหลัง Load

ใช้วิธี **Control Totals** คือคำนวณตัวเลขสำคัญจากไฟล์ CSV ต้นทางโดยตรง โดยไม่ผ่าน pipeline แล้วนำมาเทียบกับผลที่ query ออกจากคลังข้อมูล หากตัวเลขไม่ตรงกันแม้แต่ตัวเดียวแปลว่ามีข้อมูลตกหล่นหรือถูกนับซ้ำระหว่างทาง

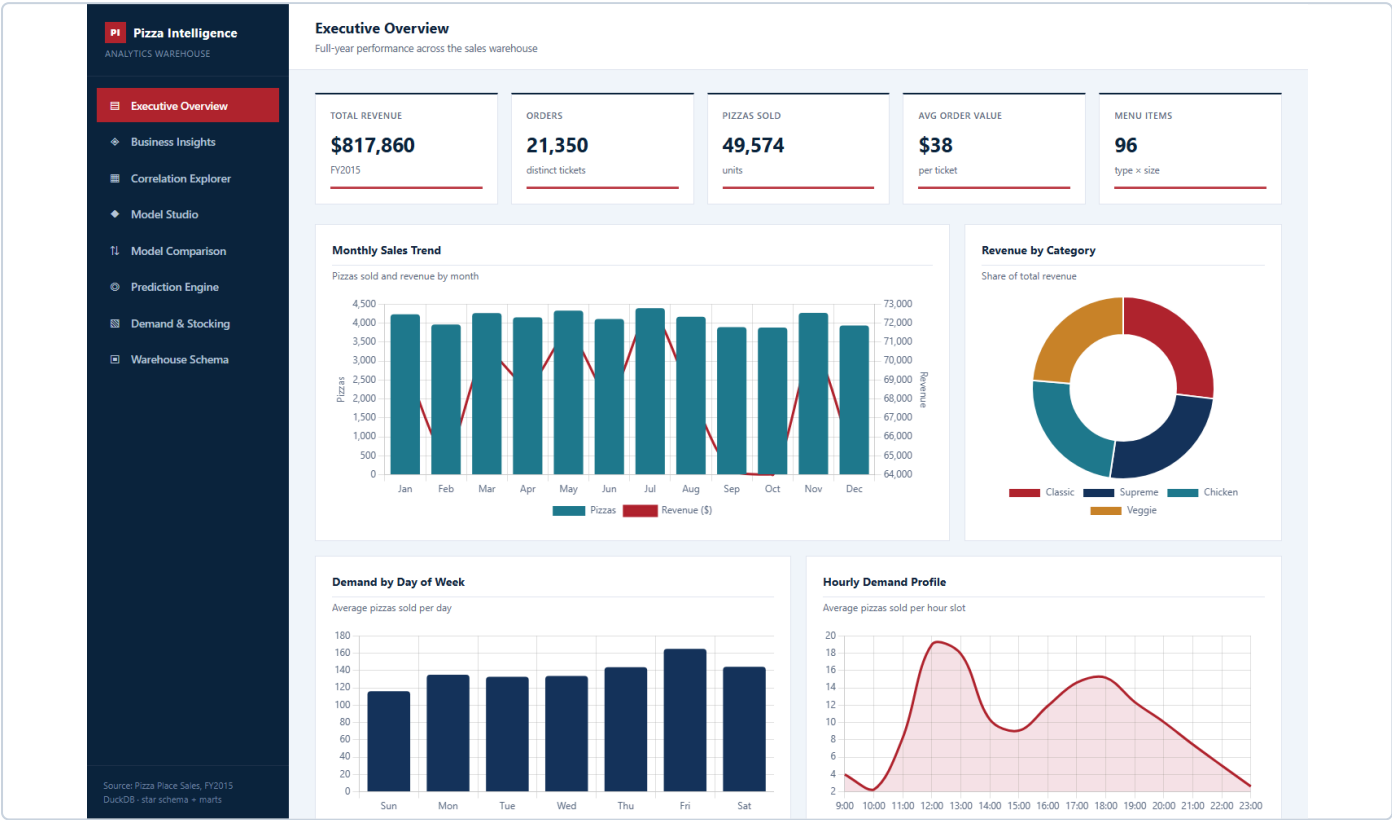
ตัวชี้วัด	คำนวณจากต้นทาง	query จากคลัง	ผลต่าง	ผล
Total Revenue (M1)	817,860.05	817,860.05	0.00	PASS
Total Quantity (M2)	49,574	49,574	0	PASS
Number of Orders (M3)	21,350	21,350	0	PASS
จำนวนแถวใน Fact Table	48,620	48,620	0	PASS

ผลการทดสอบการรันซ้ำ — Notebook มีจุดตรวจสอบอัตโนมัติ 41 จุด ครอบคลุมจำนวนแถวหลัง join ความครบถ้วนของ Foreign Key ช่วงค่าที่สมเหตุสมผล และ Control Totals หากข้อใดไม่ผ่าน pipeline จะหยุดทันทีแทนที่จะปล่อยข้อมูลผิดเข้าคลัง

ผลการรันล่าสุด ผ่านทั้ง 41 จุด และเมื่อรันซ้ำจาก checkout ใหม่ ได้ไฟล์คลังข้อมูลที่ เหมือนเดิมทุกไบต์ (ค่า hash ตรงกัน) ยืนยันว่ากระบวนการเป็น Idempotent อย่างแท้จริง

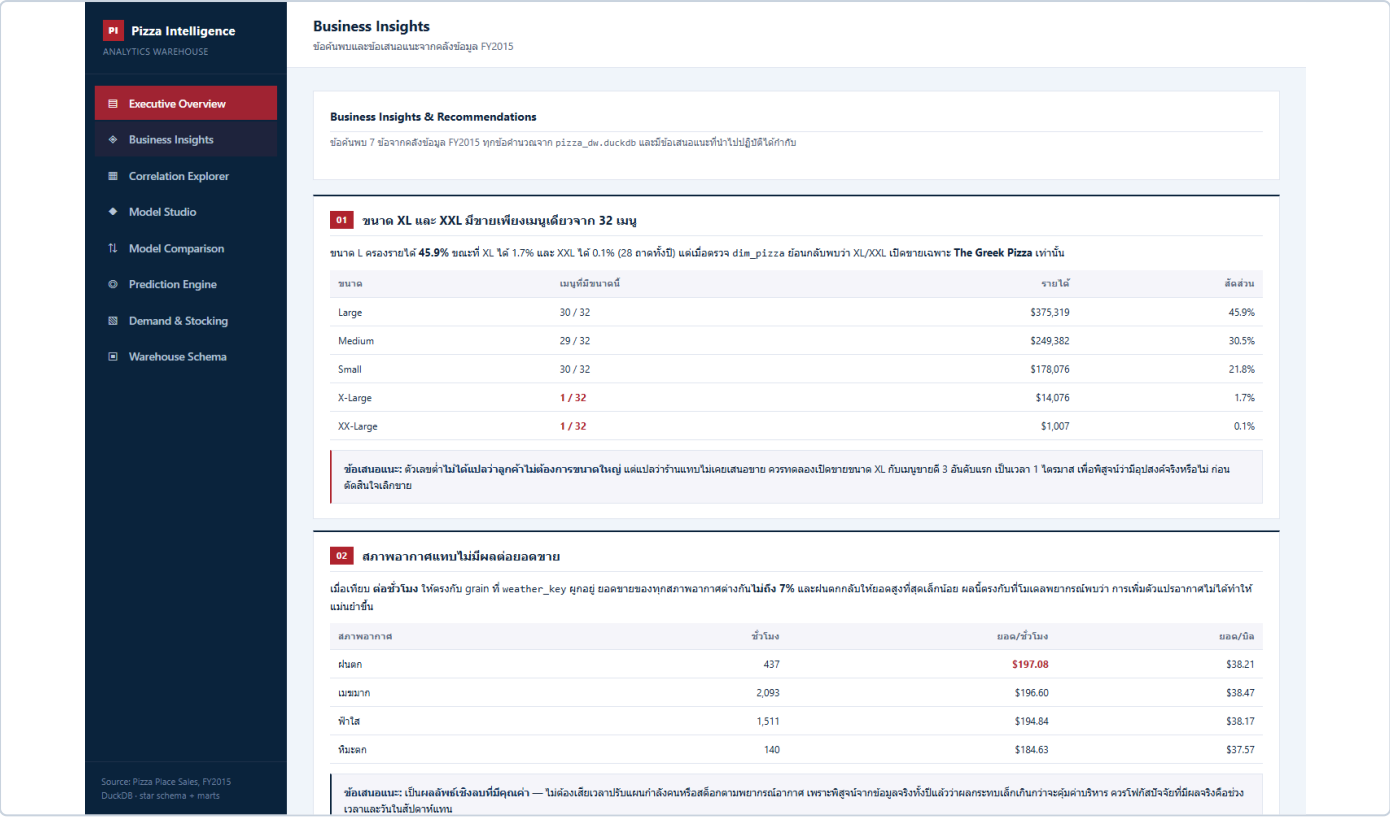
7. Dashboard

Dashboard พัฒนาเป็นไฟล์ HTML เดียวที่เปิดด้วยเบราว์เซอร์ได้ทันที โดยดึงข้อมูลทั้งหมดมาจากคลัง `pizza_dw.duckdb` ตัวไฟล์ถูกสร้างขึ้นจากเทมเพลตด้วยสคริปต์ใน `pipeline/` และเผยแพร่เป็น `index.html` ผ่าน GitHub Pages เพื่อให้เปิดใช้งานผ่านลิงก์ได้ทันที ประกอบด้วย 8 หน้าจอ ได้แก่ Executive Overview, **Business Insights**, Correlation Explorer, Model Studio, Model Comparison, Prediction Engine, Demand & Stocking และ Warehouse Schema



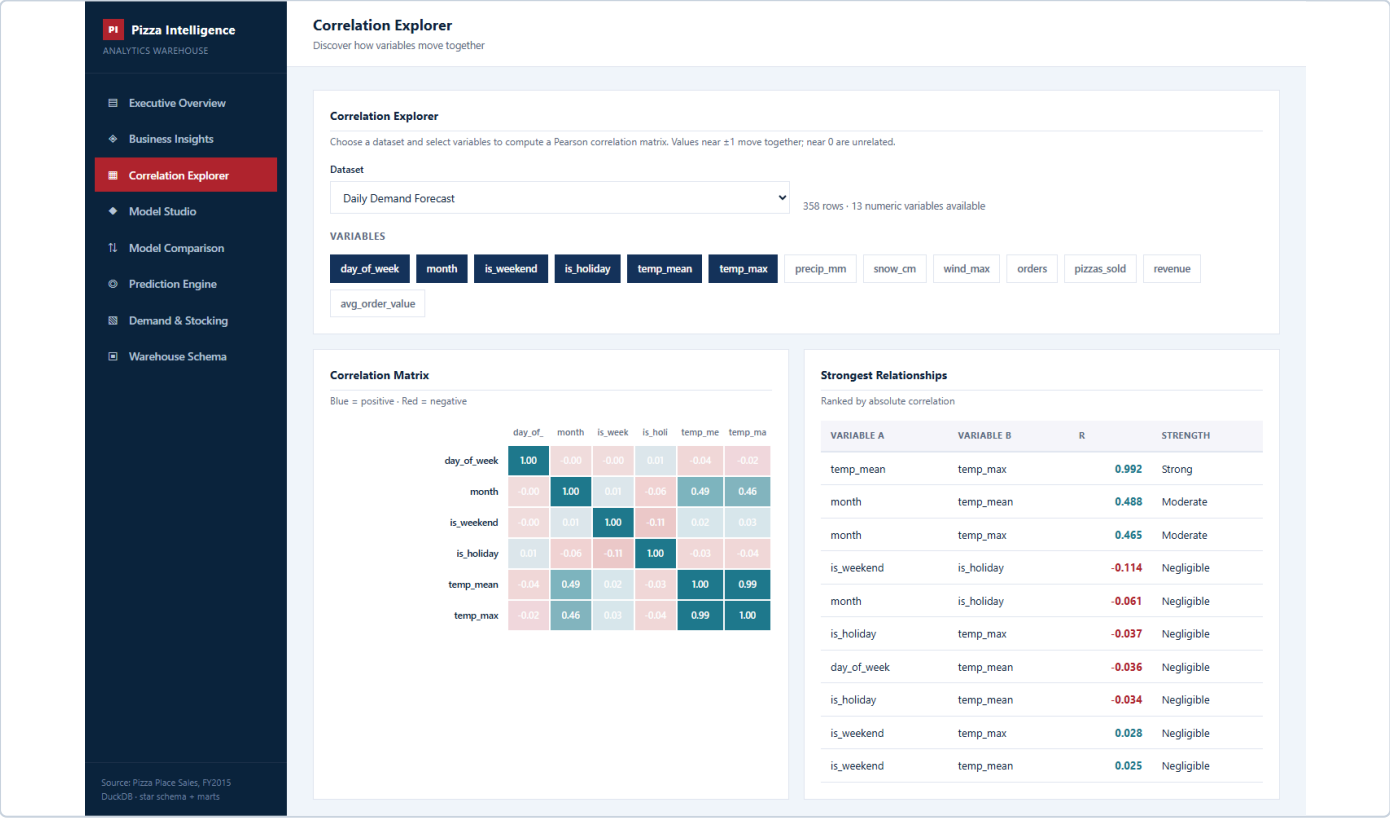
ภาพที่ 2 — หน้า Executive Overview แสดง Measure สำคัญ 5 ตัว พร้อมกราฟแนวโน้มรายเดือน สัดส่วนตามหมวด ความต้องการรายวัน และโปรไฟล์รายชั่วโมง

องค์ประกอบ	จำนวน	รายละเอียด
Measure สรุปผลประกอบการ	5 ตัว	Total Revenue, Orders, Pizzas Sold, Avg Order Value, Menu Items
กราฟทั้งหมด	มากกว่า 10 กราฟ	กระจายอยู่ใน 8 หน้าจอ
การวิเคราะห์ตามช่วงเวลา	มี	Monthly Sales Trend (แท่ง + เส้นซ้อน) และ Hourly Demand Profile
การเปรียบเทียบข้อมูล	มี	Demand by Day of Week, Revenue by Category, Model Comparison
Filter / Interactive Control	4 รายการ	ดูรายละเอียดในหัวข้อ 7.1
Insight และข้อเสนอแนะบน Dashboard	7 ข้อ	หน้า Business Insights (ภาพที่ 3)



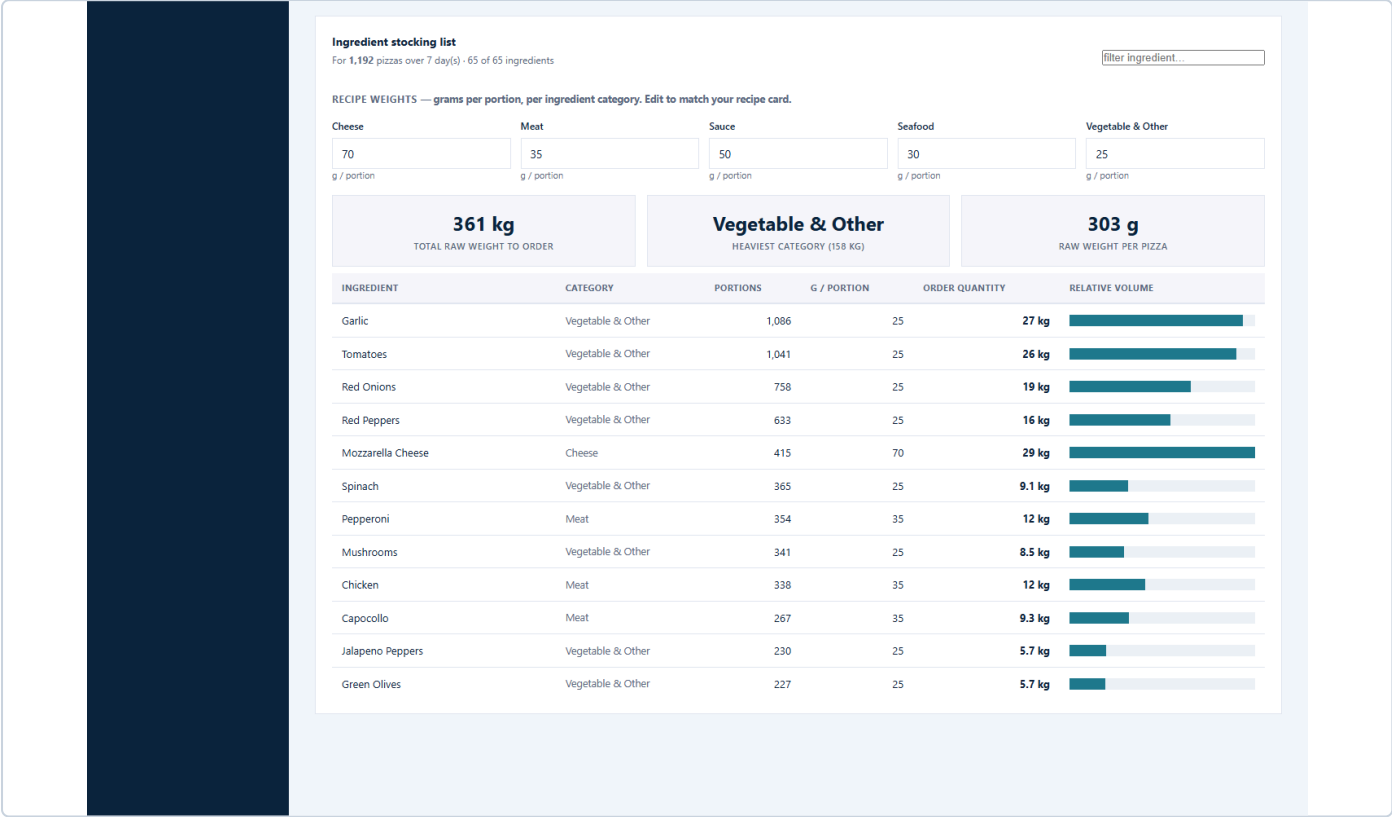
ภาพที่ 3 — หน้า Business Insights บน Dashboard (แสดง 2 ข้อแรกจากทั้งหมด 7 ข้อ)
แต่ละข้อมีตารางตัวเลขจากคลังข้อมูลและกล่องข้อเสนอนำทางธุรกิจกำกับ รายละเอียดครบทุกข้ออยู่ในหัวข้อที่ 8

7.1 ตัวอย่างการใช้ Filter และ Interactive Control



ภาพที่ 4 — หน้า Correlation Explorer สาธิตการใช้ Interactive Control ผู้ใช้เลือกชุดข้อมูลจาก Dropdown และเลือก/ยกเลิกตัวแปรได้อิสระ เมทริกซ์สหสัมพันธ์และตารางความสัมพันธ์ทางขวางจะคำนวณใหม่ทันที

Control	ประเภท	ผลการแสดงผล
Dataset Selector	Dropdown	สลับชุดข้อมูลระหว่าง Daily Demand Forecast, Hourly Demand และ Pizza Revenue
Variable Chips	ปุ่มเลือกหลายค่า	เลือกตัวแปรที่จะเข้าเมทริกซ์สหสัมพันธ์ กดสลับเปิด/ปิดได้ที่ละตัว
Prediction Engine	Slider / ฟอรั่ม	ปรับค่าตัวแปรนำเข้าแล้วดูผลพยากรณ์ความต้องการทันที
Demand & Stocking	ตัวเลือกวันที่ + แกดเลื่อน	เลือกวันที่เป้าหมาย ระบบอ่านวันในสัปดาห์ เดือน และวันหยุดให้อัตโนมัติ พร้อมเติมค่าอากาศเฉลี่ยของเดือนนั้น ซึ่งลากปรับเองได้ 4 ตัวแปร
Recipe Weights	ช่องกรอกตัวเลข	แก้ค่ากรัมต่อ portion ของวัตถุดิบแต่ละหมวด เพื่อปรับให้ตรงกับสูตรของครัวจริง ตารางสั่งซื้อคำนวณใหม่ทันที



ภาพที่ 5 — ตารางสั่งวัตถุดิบจากหน้า Demand & Stocking

แปลงยอดพยากรณ์เป็นน้ำหนักที่ต้องสั่งจริงเป็นกิโลกรัม โดยถ่วงน้ำหนักตามขนาดพิซซ่า ผ่าน Bridge Table แล้วคูณด้วยค่ากรัมต่อ portion ของแต่ละหมวดวัตถุดิบ ซึ่งผู้ใช้แก้ไขเองได้

8. Business Insights และข้อเสนอแนะ

\$817,860 TOTAL REVENUE	49,574 PIZZAS SOLD	21,350 ORDERS	\$38.31 AVG ORDER VALUE	358 TRADING DAYS
---	--	-------------------------------------	---	--

Insight 1 — ขนาด XL และ XXL มีขายเพียงเมนูเดียวจาก 32 เมนู

สิ่งที่พบ: ขนาด L ครองรายได้ 45.9% ขณะที่ XL ได้เพียง 1.7% และ XXL ได้ 0.1% (28 ถาดทั้งปี) เมื่อตรวจสอบตาราง `dim_pizza` ย้อนกลับพบว่าขนาด XL และ XXL เปิดขายเฉพาะ The Greek Pizza เท่านั้น

ขนาด	จำนวนเมนูที่มีขนาดนี้	รายได้	สัดส่วน	จำนวนถาด
Small	30 / 32	\$178,076	21.8%	14,403
Medium	29 / 32	\$249,382	30.5%	15,635
Large	30 / 32	\$375,319	45.9%	18,956
X-Large	1 / 32	\$14,076	1.7%	552
XX-Large	1 / 32	\$1,007	0.1%	28

ข้อเสนอแนะ — ตัวเลขที่ต่ำไม่ได้แปลว่าลูกค้าไม่ต้องการขนาดใหญ่ แต่แปลว่าร้านแทบไม่เคยเสนอขาย ข้อเสนอว่า “ควรเลิกขายขนาดใหญ่” จึงเป็นข้อสรุปที่ผิด

สิ่งที่ควรทำ: ทดลองเปิดขายขนาด XL กับเมนูขายดี 3 อันดับแรก (Thai Chicken, Barbecue Chicken, California Chicken) เป็นเวลา 1 ไตรมาส เพื่อพิสูจน์ว่าอุปสงค์มีจริงหรือไม่ก่อนตัดสินใจ

Insight 2 — สภาพอากาศแทบไม่มีผลต่อยอดขาย

สภาพอากาศ	จำนวนชั่วโมง	ยอดขายต่อชั่วโมง	ยอดขายต่อบิล
ฝนตก	437	\$197.08	\$38.21
เมฆมาก	2,093	\$196.60	\$38.47
ฟ้าใส	1,511	\$194.84	\$38.17
หิมะตก	140	\$184.63	\$37.57

สิ่งที่พบ: ยอดขายต่อชั่วโมงของทุกสภาพอากาศต่างกันไม่ถึง 7% และฝนตกกลับให้ยอดสูงที่สุดเล็กน้อย ส่วนยอดขายต่อบิลแทบไม่ขยับเลย

ข้อเสนอแนะ — เป็นผลลัพธ์เชิงลบที่มีคุณค่า ผู้บริหารไม่จำเป็นต้องเสียเวลาปรับแผนกำลังคนหรือสต็อกตามพยากรณ์อากาศ เพราะพิสูจน์จากข้อมูลจริงทั้งปีแล้วว่าผลกระทบเล็กน้อยกว่าจะคุ้มค่าบริหาร ควรนำเวลาไปโฟกัสกับปัจจัยที่มีผลจริงคือช่วงเวลาและวันในสัปดาห์แทน

Insight 3 — วันหยุดนักขัตฤกษ์ขายดีกว่าวันธรรมดา แต่วันอาทิตย์แย่มาก

ประเภทวัน	วัน	ยอด/วัน	วัน	ยอด/วัน
วันหยุดนักขัตฤกษ์	11	\$2,637	ศุกร์	\$2,721
วันธรรมดา	243	\$2,331	พฤหัสบดี	\$2,376
สุดสัปดาห์	104	\$2,138	เสาร์	\$2,369
			จันทร์-พุธ	~\$2,210
			อาทิตย์	\$1,908

สิ่งที่พบ: วันหยุดนักขัตฤกษ์ขายยอดสูงกว่าวันธรรมดา 13% และสูงกว่าสุดสัปดาห์ถึง 23% ขณะที่วันอาทิตย์ขายยอดต่ำที่สุดในรอบสัปดาห์ ต่ำกว่าวันศุกร์ซึ่งเป็นวันที่ดีที่สุดถึง 30%

ข้อเสนอแนะ — จัดกำลังคนเพิ่มในวันหยุดนักขัตฤกษ์และวันศุกร์ และออกโปรโมชั่นเจาะจงวันอาทิตย์ เช่น ชุดครอบครัวราคาพิเศษ เพื่อดึงยอดวันที่อ่อนที่สุดขึ้นมา หากยกวันอาทิตย์ให้เท่าค่าเฉลี่ยได้ จะเพิ่มรายได้ราว \$22,000 ต่อปี

Insight 4 — รายได้ 70% กระจุกอยู่ในสองช่วงเวลา

ช่วงเวลา	จำนวนบิล	รายได้	สัดส่วน
Dinner (17:00–20:59)	8,386	\$306,379	37.5%
Lunch (11:00–13:59)	6,206	\$262,879	32.1%
Afternoon (14:00–16:59)	4,860	\$182,249	22.3%
Late Night (21:00–23:59)	1,889	\$65,966	8.1%
Morning (09:00–10:59)	9	\$387	0.0%

สิ่งที่พบ: ชั่วโมงที่ทำรายได้สูงสุดคือ 12:00–12:59 (\$111,878) รองลงมาคือ 13:00–13:59 (\$106,066) ส่วนพีครองคือมือเย็น 17:00–19:00 ขณะที่ช่วง 09:00–10:59 มีเพียง **9 บิล** ตลอดทั้งปี

ข้อเสนอแนะ — จัดกะพนักงานให้หนาแน่นช่วง 11:00–14:00 และ 17:00–20:00 ซึ่งรวมกันสร้างรายได้เกือบ 70% และ **พิจารณาเลื่อนเวลาเปิดร้านจาก 09:00 เป็น 11:00** เพราะสองชั่วโมงแรกสร้างรายได้เพียง \$387 ตลอดปี ไม่คุ้มค่าแรงและค่าสาธารณูปโภค

Insight 5 — ช่องว่างระหว่างเมนูขายดีกับขายไม่ออกสูงถึง 3.7 เท่า

5 อันดับแรก

เมนู	รายได้
Thai Chicken	\$43,434
Barbecue Chicken	\$42,768
California Chicken	\$41,410
Classic Deluxe	\$38,181
Spicy Italian	\$34,831

5 อันดับสุดท้าย

เมนู	รายได้
Brie Carre	\$11,588
Green Garden	\$13,956
Spinach Supreme	\$15,278
Mediterranean	\$15,361
Spinach Pesto	\$15,596

สิ่งที่พบ: The Brie Carre Pizza ขายได้เพียง 490 ถาดตลอดปี ห่างจากอันดับหนึ่งถึง 3.7 เท่า และเป็นเมนูเดียวที่รายได้ต่ำกว่า \$12,000

ข้อเสนอแนะ — พิจารณาลด Brie Carre ออกจากเมนู เพื่อลดภาระวัตถุดิบเฉพาะทางที่ใช้กับเมนูอื่นไม่ได้ **แต่ต้องตรวจสอบอัตรากำไรก่อนตัดสินใจ** เพราะเมนูราคาสูงที่ขายน้อย อาจยังทำกำไรต่อถาดได้ดี ซึ่งต้องรอ Measure M7 ที่ยังคำนวณไม่ได้ในขณะนี้

Insight 6 — ร้านปิดทุกวันจันทร์ตลอดเดือนตุลาคม

วันที่ปิด	วัน	หมายเหตุ
24–25 กันยายน	พฤหัสบดี–ศุกร์	ไม่ตรงกับวันหยุดใด
5, 12, 19, 26 ตุลาคม	วันจันทร์ทั้ง 4 สัปดาห์	ครบทุกวันจันทร์ของเดือน
25 ธันวาคม	ศุกร์	Christmas Day

สิ่งที่พบ: ร้านเปิดขายจริง 358 จาก 365 วัน รูปแบบการปิดทุกวันจันทร์ต่อเนื่อง 4 สัปดาห์เป็นระบบเกินกว่าจะเกิดจากความบังเอิญ มีความเป็นไปได้สองทางคือร้านมีนโยบายหยุดชั่วคราว หรือ **ข้อมูลเดือนนั้นสูญหายตอนส่งออกจากระบบ POS**

ข้อเสนอแนะ — ตรวจสอบกับฝ่ายปฏิบัติการว่าเป็นการปิดจริงหรือข้อมูลหาย และไม่ว่าจะเป็นกรณีใด ต้องใช้ **M6 (ยอดขายต่อวันทำการ)** แทนยอดรวมรายเดือนในการเปรียบเทียบ เพราะเดือนตุลาคมมียอดรวมต่ำที่สุด (\$64,028) แต่เมื่อคิดต่อวันทำการกลับอยู่อันดับ 2 (\$2,371) การดูยอดรวมดิบจะสรุปผิดทันที

Insight 7 — วัตถุดิบกระจุกตัวสูง ต้องบริหารความเสี่ยง

วัตถุดิบ	หมวด	ใช้ในกี่เมนู (จาก 32)	สัดส่วน
Garlic	ผัก	20	62.5%
Tomatoes	ผัก	18	56.3%
Red Onions	ผัก	13	40.6%
Red Peppers	ผัก	10	31.3%
Spinach	ผัก	8	25.0%

สิ่งที่พบ: กระเทียมถูกใช้ใน 20 จาก 32 เมนู หรือคิดเป็น 62.5% หากขาดวัตถุดิบชนิดนี้เพียงอย่างเดียว จะกระทบเมนูเกินครึ่งร้านทันที

ข้อเสนอแนะ — กำหนดให้กระเทียมและมะเขือเทศเป็น **วัตถุดิบวิกฤต** ที่ต้องมี Safety Stock สูงกว่าปกติ และควรมีผู้จัดจำหน่ายสำรองอย่างน้อย 2 ราย ขณะที่วัตถุดิบเฉพาะทางที่ใช้เมนูเดียวควรสั่งแบบ Just-in-time เพื่อลดของเสีย

9. การใช้ Generative AI และการตรวจสอบผลลัพธ์

กลุ่มใช้ Generative AI ช่วยในขั้นตอนวิเคราะห์ Requirement ออกแบบ Data Model เขียนโค้ด Python และ SQL รวมถึงช่วยอธิบาย Insight โดยยึดหลักว่า ทุกผลลัพธ์ที่ AI สร้างต้องถูกตรวจสอบย้อนกลับไปหาข้อมูลจริงก่อนนำมาใช้

9.1 Prompt สำคัญที่ใช้

#	วัตถุประสงค์	สาระสำคัญของ Prompt
1	วิเคราะห์ Requirement	ให้ตั้ง Business Problem, Stakeholders, Business Questions และ Measures จาก dataset ที่มี โดยกำชับว่าขอเฉพาะคำถามที่ข้อมูลชุดนี้ตอบได้จริง
2	หาแหล่งข้อมูลเพิ่ม	ขอคำแนะนำแหล่งข้อมูลฟรีที่ไม่ต้องใช้ API key และเชื่อมกับยอดขายรายชั่วโมงปี 2015 ได้ พร้อมเขียนสคริปต์ดึงข้อมูลที่รันซ้ำได้และมี retry
3	ตรวจสอบคุณภาพข้อมูล	ให้เขียนโค้ด profile ข้อมูล 3 แหล่ง ตรวจสอบค่าว่าง แยกซ้ำ คีย์ที่ join ไม่ตรง รูปแบบวันที่ และหน่วยวัด โดยยังไม่ต้องแก้เพื่อเก็บหลักฐานก่อนแก้
4	ออกแบบ Star Schema	ให้ออกแบบ Fact 1 ตารางและ Dimension อย่างน้อย 5 ตาราง ระบุ Grain, PK, FK และอธิบายเหตุผลของการแยก Dimension
5	เขียน ETL Notebook	ให้เขียน Notebook ครอบคลุม Extract ถึง Load เข้า DuckDB โดยห้ามแก้ไฟล์ต้นทาง ต้องรันซ้ำได้ มี Audit Log และ Control Totals

9.2 สิ่งที่ AI แนะนำและกลุ่มนำไปใช้

- แยก Measure เป็น Base และ Derived พร้อมเหตุผลว่าค่าเฉลี่ยนำมาบวกกันไม่ได้ — นำไปใช้
- ใช้ Mini-Dimension สำหรับสภาพอากาศโดยจัดค่าต่อเนื่องเป็นช่วง — นำไปใช้ ผลคือเหลือ 23 แถวจาก 280 combination
- สร้าง dim_date จากปฏิทินเต็มปีแล้วเพิ่มอง is_trading_day — นำไปใช้
- เตือนว่า Bridge Table ห้ามใช้รวมยอดขายเพราะจะนับซ้ำ — นำไปใช้ และระบุไว้ในเอกสาร
- เสนอให้ ไม่แก้ ปัญหา DST แต่บันทึกเป็นข้อจำกัด — นำไปใช้

9.3 วิธีตรวจสอบว่าผลลัพธ์จาก AI ถูกต้อง

วิธีที่ 1 — รันจริงทั้งไฟล์ ไม่ใช่แค่อ่านโค้ด

การรันครั้งแรกเกิด TypeError: unhashable type: 'list' เพราะข้อมูลจาก API มีคอลัมน์ที่เก็บ list ซึ่งเรียก duplicated() ไม่ได้ กลุ่มแก้ไขโดยแปลงคอลัมน์ดังกล่าวเป็น string ก่อนตรวจแถวซ้ำ

วิธีที่ 2 — เปรียบตัวเลขกับที่คำนวณเองจากไฟล์ต้นทาง

กลุ่มคำนวณยอดขายรวมจาก CSV ดิบด้วย pandas แยกต่างหาก ได้ \$817,860.05 แล้วเทียบกับที่ query จาก DuckDB พบว่าตรงกันทุกหลัก

วิธีที่ 3 — ตรวจสอบข้อสรุปย้อนกลับไปหาข้อมูลดิบ จนพบว่า AI สรุปลิด

SQL ที่ AI เขียนเพื่อตอบคำถามเรื่องสภาพอากาศ หายอดขายด้วยจำนวนวัน ได้ผลว่าฝนตกทำให้ยอดขายลดลง 39% ซึ่งดูน่าเชื่อถือมาก

สภาพอากาศ	ผลที่ AI ให้ (ต่อวัน)	ผลที่ถูกต้อง (ต่อชั่วโมง)
เมฆมาก	\$1,344.74	\$196.60
ฟ้าใส	\$1,263.52	\$194.84
ฝนตก	\$820.21	\$197.08
หิมะตก	\$760.23	\$184.63

กลุ่มตรวจสอบโดยบวกจำนวนวันจากทุกกลุ่ม ได้ 678 วัน ทั้งที่ทั้งปีมีวันทำการจริงเพียง 358 วัน แสดงว่าตัวหารนับวันซ้ำข้ามกลุ่ม เนื่องจากสภาพอากาศเปลี่ยนได้ทุกชั่วโมง วันหนึ่งจึงมีหลายสภาพอากาศ วันฝนตกส่วนใหญ่ฝนตกเพียงไม่กี่ชั่วโมง ตัวเศษจึงเป็นรายได้เฉพาะชั่วโมงนั้น แต่ตัวหารกลับเป็นทั้งวัน

บทเรียนสำคัญ — AI เขียน SQL ที่ถูกไวยากรณ์และรันผ่านโดยไม่มี error แต่ Grain ของตัวหารไม่ตรงกับ Grain ของข้อมูล ซึ่งเป็นความผิดพลาดที่ไม่มีระบบใดฟ้อง และให้ผลลัพธ์ที่ดูสมเหตุสมผลจนน่าเชื่อ หากไม่ตรวจสอบย้อนกลับไปหาข้อมูลดิบจะไม่มีทางจับได้เลย เมื่อแก้เป็นหารด้วยจำนวนชั่วโมงแล้ว ข้อสรุปกลับด้านทันที

วิธีที่ 4 — ใส่ Assertion ให้ Pipeline ตรวจสอบตัวเองทุกครั้งที่รัน

Notebook มีจุดตรวจ 41 จุด ครอบคลุมจำนวนแถวหลัง join ความครบถ้วนของ FK ช่วงค่าที่สมเหตุสมผล และ Control Totals หากข้อใดไม่ผ่าน pipeline จะหยุดทันที

10. สรุปและลิงก์ไปยังไฟล์ประกอบ

10.1 สรุปผลการดำเนินโครงการ

ข้อกำหนด	เป้าหมาย	ที่ทำได้	หมายเหตุ
แหล่งข้อมูล	อย่างน้อย 3	3	2 แหล่งเป็น REST API
แหล่งที่ซับซ้อนกว่า CSV	อย่างน้อย 1	2	Nested JSON ทั้งคู่
ปัญหาคุณภาพข้อมูล	อย่างน้อย 3 ประเภท	7	มี Audit Log ก่อน-หลังทุกข้อ
Fact Table	อย่างน้อย 1	1	Transaction Fact 48,620 แถว
Dimension Tables	อย่างน้อย 5 (มี Date)	5 + Bridge	มี <code>dim_date</code> ครบตามกำหนด
Measures	อย่างน้อย 3	6 พร้อมใช้	M7 รอข้อมูลต้นทุน
กราฟบน Dashboard	อย่างน้อย 5	มากกว่า 10	กระจายใน 7 หน้าจอ
Filter / Interactive Control	อย่างน้อย 2	4	Dropdown, Chips, Slider, ช่องกรอก
Business Insights	อย่างน้อย 5	7	อยู่บน Dashboard และในรายงาน

10.2 ข้อเสนอแนะสำหรับการพัฒนาต่อ

- จัดทำตารางต้นทุนวัตถุดิบ เพื่อปลดล็อก M7 ถ้าไรชั้นต้น ซึ่งจะทำให้ตอบได้ว่าควรดันหรือตัดเมนูใดอย่างมีหลักฐาน
- เพิ่ม **Fact Table** ที่สอง ระดับการใช้วัตถุดิบรายวัน เพื่อวิเคราะห์ของเสียและประสิทธิภาพการสั่งซื้อ
- ตรวจสอบข้อมูลเดือนตุลาคม ว่าวันจันทร์ที่หายไปเป็นการปิดจริงหรือข้อมูลสูญหาย
- เก็บรหัสลูกค้าในระบบ POS เพื่อเปิดทางให้วิเคราะห์การกลับมาซื้อซ้ำในอนาคต

10.3 ลิงก์ไปยังไฟล์ประกอบ

รายการ	ลิงก์
โฟลเดอร์ Source Files (Google Drive)	ตั้งค่าเป็น Anyone with the link can view
Dashboard	ไฟล์ <code>Pizza_Sales_Dashboard.html</code>
วิดีโอแนะนำ	

10.4 โครงสร้างโฟลเดอร์ Source Files

โฟลเดอร์ / ไฟล์	เนื้อหา
01_Raw_Data/	CSV ยอดขาย 4 ไฟล์, JSON สภาพอากาศและวันหยุด, <code>SOURCES.md</code> , manifest พร้อม SHA-256
02_ETL/	<code>fetch_sources.py</code> ดึงข้อมูลจาก API และ <code>etl_pipeline.ipynb</code> กระบวนการ ETL ทั้งหมด
03_Data_Warehouse/	<code>pizza_dw.duckdb</code> และ CSV ส่งออก 7 ไฟล์
04_Dashboard/	<code>Pizza_Analysis.ipynb</code> สำหรับ EDA และ Machine Learning
<code>index.html</code>	ไฟล์ Dashboard ที่เผยแพร่ผ่าน GitHub Pages วางไว้ที่รากของโปรเจกต์เพราะ GitHub Pages ให้บริการจากรากหรือ <code>/docs</code> เท่านั้น
pipeline/	สคริปต์สร้าง Dashboard และฝึกโมเดล พร้อมชุดทดสอบ <code>test_stocking.py</code>
05_AI_Usage_Log/	บันทึก Prompt ที่ใช้ สิ่งที่ AI แนะนำ และวิธีตรวจสอบผลลัพธ์
README	ภาพรวมโครงการและวิธีรันทุกขั้นตอน